Lab: 12

Pointers in C++

Objectives

In this lab we will learn:

- Computer Memory and Variables
- Pointers
- Pointers Declaration
- Referencing Operator
- Referencing Operator

1: Computer Memory

Essentially, the computer's memory is made up of bytes. Each byte has a number, an address, associated with it. The picture below represents several bytes of a computer's memory. In the picture, addresses 924 thru 940 are shown.

1.1: Variable and Computer Memory

A variable in a program is something with a name, the value of which can vary. The way the compiler handles this is that it assigns a specific block of memory within the computer to hold the value of that variable. The size of that block depends on the range over which the variable is allowed to vary. For example, on 32 bit PC's the size of an integer variable is 4 bytes. On older 16 bit PCs integers were 2 bytes.

Example:

```
#include <iostream>
using namespace std;
int main() {
float fl = 3.14;
// declare a variable of type float
cout << fl;
return 0; }
```

Output:

3.14

In this program, the computer reserves memory for fl. Depending on the computer's architecture, a float may require 2, 4, 8 or some other number of bytes. In our example, we'll assume that a float requires 4 bytes.

Be careful that : the illustration that shows 3.14 in the computer's memory can be misleading. Looking at the diagram, it appears that"3"is stored in memory location 924,"."is stored in memory location 925,"1"in 926, and"4"in 927.

Keep in mind that the computer actually converts the floating point number 3.14 into a set of ones and zeros. Each byte holds 8 ones or zeros. So, our 4 byte float is stored as 32 ones and zeros (8 per byte times 4 bytes). Regardless of whether the number is 3.14, or 273.15, the number is always stored in 4 bytes as a series of 32 ones and zeros.

2.1: Computer Memory

The syntax is as shown below.

We start by specifying the type of data stored in the location identified by the pointer. The asterisk tells the compiler that you are creating a pointer variable. Finally you give the name of the variable. Such a variable is called a pointer variable. In C++ when we define a pointer variable we do so by preceding its name with an as- terisk. In C++ we also give our pointer a type which, in this case, refers to the type of data

stored at the address we will be storing in our pointer. For example, consider the variable declaration:

Example:

```
Int *ptr;

 Int k;
```

- ptr is the name of our variable (just as k is the name of our integer variable).
- The'*'informs the compiler that we want a pointer variable, i.e. to set aside many bytes required to store an address in memory. The int says that we intend to use our pointer variable to store the address of an integer.

2.2: Referencing Operator

Suppose now that we want to store the address of our integer variable k in ptr. To do this we use the unary & operator and write:

Example:

```
int *ptr;
  int k;
   ptr = &k
```

What the & operator does (in ptr = & k;), is retrieve the address of k, and copies that to the contents of our pointer ptr. Now, ptr is said to "pointto"k.

2.3: Dereferencing Operator

The "dereferencing operator" is the asterisk and it is used as follows:

Example:

```
*ptr=7;
```

This will copy 7 to the address pointed to by ptr. Thus if ptr "pointsto"(contains the address of) k, the above statement will set the value of k to 7. That is, when we use the'*'this way, we are referring to the value of that which ptr is pointing to, not the value of the pointer itself.

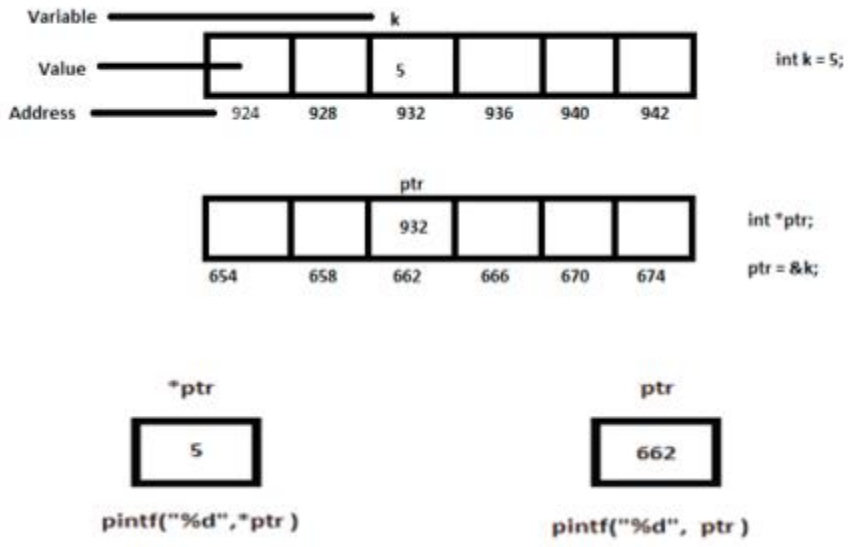
Similarly, we could write:

Example:

```
Cout<<*ptr<<endl;
```

to print to the screen the integer value stored at the address pointed to by ptr.

2.4: Graphical Representation of Pointer



Example: Variable and Computer Memory

```
#include <iostream>
using namespace std;
int main() {
int a = 5;
int b = 15;
int *p1, *p2;          // declare two pointers



p1 = &a;
```

```
// p1 = address of a
p2 = &b;

// p2 = address of b
*p1 = 10;

// value pointed by p1 is
10 *p2 = *p1;

// value pointed by p2 = value pinted by
*p1 = 20;
// value pointed by p1
}
 cout << "a = " << a << endl; cout << "b = " << b << endl;
return 0;}
```

Output:

A=20

B=10

Lab Tasks:

Task 1: Pointer Declaration and initialation

- Declare an integer variable called num and initialize it with a value of your choice.
- Declare a pointer variable called ptr of type integer.
- Initialize the pointer variable to point to the address of the num variable.
- Display the value of num and the value pointed to by ptr using the de reference operator.

```
#include <iostream>                    // Header file for input and output

using namespace std;                   // Use standard namespace

// Main function
int main()
{

// Declare an integer variable and initialize it
int num = 25;

// Declare a pointer variable of integer type
int *ptr;

// Store the address of num in pointer ptr
ptr = &num;

// Display the value of num
cout << "Value of num = " << num << endl;

// Display the address of num using pointer
cout << "Address stored in ptr = " << ptr << endl;

// Display the value pointed to by ptr using dereference operator
cout << "Value pointed by ptr = " << *ptr << endl;

// End the program successfully
return 0;
}
```

Task 2: Pointer Arithmetic

- Declare an integer array called numbers with some values of your choice.
- Declare a pointer variable called ptr of type integer and initialize it to point to the first
- element of the num array.
- Use pointer arithmetic to access and display the elements of the array using the pointer.

```
#include <iostream>          // Header file for input and output

using namespace std;         // Use standard namespace

int main()                   // Main function
        {
// Declare and initialize an integer array
int numbers[5] = {10, 20, 30, 40, 50};
int *ptr;                    // Declare a pointer variable

// Initialize pointer to point to first element of array
ptr = numbers;

// Display array elements using pointer arithmetic
cout << "Array elements using pointer arithmetic:" << endl;

// Access elements using pointer arithmetic
cout << *(ptr + 0) << endl;  // First element
cout << *(ptr + 1) << endl;  // Second element
cout << *(ptr + 2) << endl;  // Third element
cout << *(ptr + 3) << endl;  // Fourth element
cout << *(ptr + 4) << endl;  // Fifth element
return 0;                    // End the program successfully
}
```